Skip Lists: Probabilistic Search Optimization

The Problem

Searching in a regular linked list: O(n)

```
Find 77:
[10] → [23] → [34] → [42] → [56] → [77] → [88] → [99]
 ↑                                   ↑
Start                              Target (6 steps!)
```

Binary Search Trees solve this: O(log n)

- But require complex balancing (AVL, Red-Black)
- Rotation operations are tricky

Question: Can we get O(log n) search in a linked list without complex balancing?

Answer: Skip Lists!

What is a Skip List?

A **Skip List** is a probabilistic data structure that:

- Builds **multiple levels** of linked lists
- Higher levels have **fewer nodes** (skipping nodes)
- Provides **O(log n) search** on average
- Uses **randomization** instead of balancing

Think of it as: An express lane system on a highway!

Skip List Levels: Highway Analogy

```
Level 3 (Express):    [10]──────────────────→[77]────→[99]
                       ↓                      ↓        ↓
Level 2 (Fast):       [10]──→[34]────→[77]──→[88]→[99]
                       ↓      ↓        ↓      ↓    ↓
Level 1 (Normal):     [10]→[23]→[34]→[42]→[56]→[77]→[88]→[99]
                       ↓    ↓    ↓    ↓    ↓    ↓    ↓    ↓
Level 0 (All):        [10]→[23]→[34]→[42]→[56]→[77]→[88]→[99]
```

Key Idea:

- Level 0: All nodes
- Level 1: ~50% of nodes
- Level 2: ~25% of nodes
- Level 3: ~12.5% of nodes

Search: Start at top, skip ahead when possible, drop down when overshooting!

Skip List Structure

Node with Multiple Levels:

```
struct SkipNode {
    int data;
    struct SkipNode** forward;  // Array of pointers
    int level;
};

struct SkipList {
    struct SkipNode* head;
    int max_level;
    int current_level;
};
```

Visual:

```
Level 2: •────→
Level 1: •────→
Level 0: •────→
 Data: 34
```

Searching in Skip List

Find value 77:

```
Start at top-left (head, level 3)

Step 1: Level 3, at [10]
  Next is [77] - not greater than 77, move forward

Level 3: [10]──────────────────→[77] ← Found!
           ↑                     ↑
         Start                 Target

Total comparisons: 2
```

Compare with linear search: Would need 6 comparisons!

Search Algorithm

```c
struct SkipNode* search(struct SkipList* list, int target) {
    struct SkipNode* current = list->head;

    // Start from highest level
    for (int i = list->current_level; i >= 0; i--) {
        // Move forward while next < target
        while (current->forward[i] != NULL &&
               current->forward[i]->data < target) {
            current = current->forward[i];
        }

        // Check if found
        if (current->forward[i] != NULL &&
            current->forward[i]->data == target) {
            return current->forward[i];
        }
    }

    return NULL;
}
```


Time Complexity: O(log n) average case

Insertion: Random Levels

Key Question: When inserting, how many levels should a new node have?

Answer: Flip a coin!

Randomized Level Selection:

```
int randomLevel() {
    int level = 0;

    // Flip coin: 50% chance to go higher
    while (rand() % 2 == 1 && level < MAX_LEVEL) {
        level++;
    }

    return level;
}
```

Probability:

- Level 0: 100%

Insertion Example

Insert value 65:

Step 1: Search to find insertion point

```
Level 2: [10]──→[34]────────→[77]
                  ↑
        Insert 65 here
```

Step 2: Randomly decide levels (say, level = 2)

Step 3: Insert at all levels 0 through 2

```
After:
Level 2: [10]──→[34]──→[65]──→[77]
Level 1: [10]→[23]→[34]→[42]→[56]→[65]→[77]
Level 0: [10]→[23]→[34]→[42]→[56]→[65]→[77]
```

Deletion in Skip List

Delete value 34:

Before:

```
Level 2: [10]──→[34]──→[77]
Level 1: [10]→[23]→[34]→[42]→[77]
Level 0: [10]→[23]→[34]→[42]→[56]→[77]
```

After:

```
Level 2: [10]──────────────→[77]
Level 1: [10]→[23]→[42]──→[77]
Level 0: [10]→[23]→[42]→[56]→[77]
```

Update pointers at each level where 34 appears

Skip List Performance

Time Complexity

Operation	Average Case	Worst Case	Best Case
Search	O(log n)	O(n)	O(1)
Insert	O(log n)	O(n)	O(1)
Delete	O(log n)	O(n)	O(1)

Why worst case O(n)?

- Unlucky randomization could create all nodes at level 0
- Probability is extremely low (~ 1/2^n)
- In practice, behaves like O(log n)

Space Complexity

Average: O(n)

- Expected pointers: $n + n/2 + n/4 + \ldots \approx 2n$
- Small constant factor (≈ 2)

Worst case: O(n log n)

- If all nodes randomly get max height
- Extremely unlikely in practice

Skip Lists vs Binary Search Trees

Aspect	Skip List	Balanced BST
Average Search	O(log n)	O(log n)
Implementation	Simple	Complex (rotations)
Balancing	Randomized	Explicit
Concurrent Access	Easier (lock-free)	Harder
Cache Performance	Better	Worse
Deterministic?	No	Yes

Real-World Applications

Application 1: Redis Sorted Sets

Redis uses skip lists for sorted sets (ZSET):

```
ZADD leaderboard 100 "Alice"
ZADD leaderboard 95 "Bob"
ZADD leaderboard 110 "Carol"

ZRANGE leaderboard 0 -1
→ Bob, Alice, Carol (sorted)
```

Why Skip Lists?

- O(log n) insertion and retrieval
- Range queries are easy
- Simple implementation
- Good for concurrent access

Application 2: LevelDB and RocksDB

Google's LevelDB uses skip lists for **memtable**:

Workflow:

1. Writes go to skip list in memory
2. O(log n) insertion maintains sorted order
3. When full, flush to disk as sorted file
4. Skip list allows concurrent reads during writes

Benefits: Fast writes, maintains sort order, lock-free reads possible

Application 3: Concurrent Data Structures

Java's ConcurrentSkipListMap:

```
ConcurrentSkipListMap<String, Integer> map =
    new ConcurrentSkipListMap<>();

map.put("Alice", 100);  // Thread-safe!
map.put("Bob", 95);
```

Why?

- Lock-free operations possible
- Insertions don't require global restructuring
- Each level independently traversable

Summary

Key Takeaways

Sentinel Nodes:

- Special dummy nodes that guard boundaries
- Eliminate NULL checks and special cases
- Three types: single head, dual (head/tail), circular
- Used in Linux kernel, databases, Java collections

Generalized Linked Lists:

- Nodes can contain atoms OR sublists
- Represent hierarchical/nested structures
- Applications: polynomials, JSON/XML, AST
- Use union for memory efficiency

Key Takeaways (continued)

Skip Lists:

- Probabilistic data structure with multiple levels
- O(log n) search, insert, delete (average)
- Uses randomization instead of balancing
- Simpler than balanced trees
- Better for concurrent access
- Used in Redis, LevelDB, Java concurrent collections

Main Lessons:

- Sentinel nodes trade memory for simplicity
- Generalized lists handle complex hierarchical data
- Skip lists achieve O(log n) without complex balancing
- Randomization can be as powerful as deterministic algorithms